Milestone 4 – Track 1: HiCache (Hierarchical KV Cache)

CS349D, Spring 2026 – Hiva Mohammadzadeh

The milestone-3 engine evicted radix-cache leaves by dropping them: their KV pages were returned to the GPU free list and the cached prefix was gone. On multi-turn / RAG-style workloads, once the working set crosses HBM capacity, the next request that should have hit the cache instead has to re-prefill from scratch.

HiCache replaces "drop" with **demote-to-CPU**: evicted GPU pages are copied into a pinned host-memory tier, the radix node is kept in the tree marked `tier="cpu"`, and a later match against that prefix triggers a CPU->GPU **promotion** (H2D copy) before the request reads it. The CPU pool is bounded and runs its own LRU; only when both tiers can't make room does HiCache fall back to the m3 drop path.

Both the **full-credit bar** and the **`--hicache-overlap` bonus** are implemented and runnable. I'll be candid about which parts of the cliff/restore demonstration the vanilla `bench_cache.multiturn` workload exposes cleanly and which require either a re-access pattern the bench doesn't ship or deeper instrumentation than I had time to wire up.

---

1. Design

One tree, two tiers. Each `RadixNode` carries a new `tier ∈ {"gpu","cpu"}` field; its `pages` list is interpreted in that tier's index space. A node's pages live entirely in one tier – page granularity, single-tier per node, no mixing. When `cpu_pool is None` (i.e. `--cpu-cache-size-gb 0`), behavior is byte-identical to milestone 3: every m3 test stays green unchanged.

Module layout.

File	Role
`miniengine/cpu_kv_pool.py` (new)	Pinned-host mirror of `KVMemoryPool` – per-layer K and V tensors of shape `(num_pages, page_size, num_kv_heads, head_dim)`, `device="cpu"`, `pin_memory=True`. `from_budget` sizes from `--cpu-cache-size-gb`.
`miniengine/radix_cache.py` (modified)	Tier field; optional `cpu_pool`/`copy_stream`/`overlap` on `RadixCache`. `evict` demotes instead of dropping; new `_cpu_evict`, `_promote_node`, `promote_match`. Split inherits child tier (otherwise CPU-tier pages would be mis-tagged GPU on radix-tree split).
`miniengine/engine.py` (modified)	Build `CpuKvPool` when the flag is set; thread `copy_stream` (a `torch.cuda.Stream`) through to `RadixCache`. Call `promote_match` in `_setup_paged_request` before `inc_lock_ref`.
`miniengine/__main__.py` (modified)	`--cpu-cache-size-gb FLOAT` (default 0 = HiCache off), `--hicache-overlap` (flag, default off). Fail-fast validation for incompatible combos.
`miniengine/server.py` (modified)	`/cache_stats` grows an optional `hicache` subtree exposing `total_demoted_pages`, `total_promoted_pages`, `total_cpu_evicted_pages`, copy-time accumulators, and CPU pool occupancy.

Hot paths.

Demote (GPU eviction, `RadixCache._try_demote`): for each LRU-selected GPU-tier leaf, ensure CPU room (run CPU-tier LRU if `cpu_pool.num_free < need`), allocate CPU slots, copy D2H per layer, return GPU pages to the pool, repoint `node.tier="cpu"; node.pages=cpu_slots`. **Node stays in the tree** – the prefix is still cached, just colder. If even CPU eviction can't free room, fall back to dropping the GPU node entirely (m3 behavior) so eviction always makes progress.

Promote (cache hit, `RadixCache.promote_match`): walks the matched path root->leaf, finds CPU-tier nodes, and for each: allocates GPU pages (may itself trigger demotion of *other* cold nodes; fine), copies H2D, frees CPU slots, repoints `node.tier="gpu"`. The leaf is **temp-locked** during this operation so the allocator's own evict pass cannot demote anything on the path back out from under us. Finally `inc_lock_ref(match.last_node)` pins the path for the lifetime of the borrowing request – same pattern as m3.

Async overlap (`--hicache-overlap`). Off by default; the blocking path lands first. When on: - A dedicated `torch.cuda.Stream` is created at engine startup and passed to the cache. - D2H (demote) and H2D (promote) issue under `torch.cuda.stream(copy_stream)` with `non_blocking=True`. Pinned host memory (allocated by `CpuKvPool` when CUDA is available) lets these copies run as true async DMAs. - A **pending-free queue** on each pool (`deferred_free(pages, event)` + `_drain_pending_free()`) keeps source pages reserved until their recording `cuda.Event` has fired. Otherwise a fresh allocate could hand out a page that the copy stream is still reading and corrupt the in-flight DMA. As a last resort, `allocate` blocks on the oldest pending event so a caller never sees a spurious OOM when capacity is genuinely available – just not yet released. - On promote, the compute stream waits on the H2D event via `current_stream.wait_event(event)` – non-blocking on the CPU; only the GPU stream serializes – so flash-attn never reads a half-copied KV page.

---

2. Bugs the rollout surfaced

Two related m3 baseline bugs in `engine.py` came out under sustained multi-turn load. Both are double-frees:

1. **`free_paged_request`** – after a request finishes, the engine inserts `req.input_ids + req.output_ids` into the radix cache and frees the pages that came back marked `redundant`. But because `req.page_table[:n_matched] == matched_node.pages` (the same physical indices the tree's matched ancestor already owns), those "redundant" pages get added to the pool's free list while the tree still references them. When the tree later evicts that node it frees the same indices a second time, leaving phantom entries in the pool's free list.
2. **`_insert_prompt_into_cache`** – same shape, same fix. Called after every `paged_prefill_batch` and `paged_prefill_chunk`.

Symptom: under sustained multi-turn load (32+ active sessions), `pool_num_free` quietly drifts above `num_pages` (e.g. observed 117 free / 98 capacity / 57 cached = 174 phantom indices). Two requests can be handed the same page and silently corrupt each other's KV. Eventually the scheduler wedges, requests stop being admitted, the bench client times out. The first on-pass run I tried (32×5×256, conc=4) hung at request #158 because of this.

The fix is one line each: let the tree retain ownership of `redundant` indices and don't return them to the pool. (Commits `da070df` and `819eb13`.) All 59 tests pass before and after; no m3 test exercised this finish-path side effect because the post-eviction interaction with phantom free entries wasn't covered.

---

3. Correctness

Unit tests: 59/59 pass on both Mac (CPU-only) and the L4 dev VM. - `tests/test_cpu_kv_pool.py` – 11 tests for the pinned-host pool: layout, sizing math (`from_budget` floor division), alloc/free, `CpuKvOutOfMemory` on exhaustion, and a bitwise demote->promote round-trip on layout-compatible tensors. - `tests/test_radix_cache_hicache.py` – 11 tests: evict-with-cpu-pool demotes in place; bitwise KV preservation across demote+promote; `promote_match` no-op when no CPU-tier nodes on the path or when

cpu_pool is None; CPU overflow drops the LRU CPU leaf; locked nodes never demoted or dropped; fallback drop when both pools are full; num_evictable_pages tier-filter; radix-split inherits child tier; reset routes pages to the owning pool. - tests/test_pending_free.py – 6 tests: deferred_free with already-fired vs unfired events, sync-on-pending shortage, explicit drain accounting. Uses a duck-typed FakeEvent so the suite runs without CUDA.

Production smoke (L4, Qwen3-8B): the demos in section 4 ran 768 requests under HiCache without an error, with pool_num_free + num_cached_pages ≤ num_pages checked throughout (post-fix).

MMLU within ±1pp: I had time to set this up but not run it on the L4 by the deadline. The token-identity argument carries the load: demote + promote is a bitwise round-trip of K and V indexed copies, the bitwise property is covered by test_indexed_copy_preserves_kv_bitwise and test_match_then_promote_refreshes_pages_to_gpu_w
The serving path doesn't add any non-deterministic step that HiCache could poison; with greedy sampling, output token streams from HiCache-on and HiCache-off configurations should be identical on the same prompts.

---

4. Quantitative evaluation

4.1 Setup

- **Hardware:** single NVIDIA L4 (23 GB HBM), 60 GB host RAM, no swap.
- **Model:** Qwen/Qwen3-8B in float16. Weights ≈ 16 GB, leaving roughly 3.7 GB for the KV pool at --mem-fraction-static 0.85.
- **Engine config:** --mode paged --page-size 256 --prefill-chunk-size 512. Page size **must be a multiple of 256** on this hardware – flash-attn 2.x's paged kernel rejects smaller pages on Ada with *"Paged KV cache block size must be divisible by 256"*. The milestone example uses --page-size 32, which crashes at the first prefill on the L4. (Same gotcha m3 documented.)
- **GPU KV pool: 98 pages × 256 tokens = 25 088 tokens cached capacity.**
- **CPU KV tier: --cpu-cache-size-gb 24 -> 635 slots × 256 tokens ≈ 162 000 cached tokens, 6.5× the GPU pool.** Pinned host memory. *Why 24 GB and not 40 GB (~10×)?* The L4 has no swap and 60 GB total RAM; pinning 40 GB tipped the box into an OOM-kill during boot on the first attempt (see §5). 24 GB lands the CPU tier safely under the host-RAM ceiling while still giving a healthy multiple over HBM.

4.2 Full-credit demo – multiturn with eviction pressure

bench_cache.py --workload multiturn --num-sessions 128 --turns-per-session 6 --max-tokens 192 --concurrency 32, run twice with the same binary: once with HiCache off (--cpu-cache-size-gb 0, the milestone-3 baseline path) and once with HiCache on (--cpu-cache-size-gb 24). 768 requests per pass.

Totals:

Metric	OFF (m3 baseline)	ON (HiCache, blocking)
Wall time	205.1 s	202.6 s
Throughput	3.74 req/s	3.79 req/s
Generation rate	163 tok/s	159 tok/s
Overall hit rate	32.7%	32.0%
Pages inserted	291	278
GPU pages evicted (dropped)	**192**	**0**
Pages demoted GPU->CPU	–	**200**
Pages promoted CPU->GPU	–	**7**
Final num_cached_pages	99	85 (GPU) + 193 (CPU)
CPU pool occupancy at end	–	193 / 635 slots

Per-turn breakdown (averaged across all 128 sessions):

turn	OFF hit_rate	OFF TTFT_p50	ON hit_rate	ON TTFT_p50
0	0.0%	491 ms	0.0%	476 ms
1	0.0%	521 ms	0.0%	508 ms
2	0.0%	405 ms	0.0%	496 ms
3	25.7%	452 ms	18.0%	477 ms
4	56.3%	355 ms	60.7%	319 ms
5	61.8%	324 ms	61.2%	358 ms

What the numbers show. - **GPU drops are eliminated.** Off-pass dropped 192 pages by the end of the run; HiCache moved those exact would-be-drops (and 8 more from continued pressure) into the CPU tier and **dropped zero**. The mechanism is working. - **Promotions happened (7).** Even on this workload – which I argue below is not the easy case for HiCache – there were 7 re-access events where a request matched a prefix that had already been demoted to CPU, and HiCache brought it back from host memory instead of forcing a re-prefill. - **Overall hit rate and throughput are essentially the same.** This is the fair part of "what didn't work": vanilla `bench_cache.multiturn` is **not** the workload that maximally rewards HiCache. The bench's worker pool pulls a session off the queue, runs *all* its turns sequentially, then picks the next session. So each session's prior-turn prefix is the most recently cached thing right when the next turn looks for it – it's never the LRU victim. Eviction (whether drop or demote) hits prefixes belonging to *finished* sessions, which the workload won't access again. So the per-turn hit rate climbs identically in both passes, and the wall-clock saving from the 7 successful promotions is tiny relative to total run time.

4.3 The cliff that vanilla `bench_cache.multiturn` doesn't expose

The milestone spec describes a per-turn hit-rate cliff: ≥70% on turn 1, <20% by the last turn. I couldn't reproduce that shape on this workload because of the worker-pool access pattern above. I tried two variants:

1. **`conc=NUM_SESSIONS` lockstep** (32×6, conc=32): all 32 sessions advance their turns roughly in step. Cache never overflowed (75 cached / 98 capacity), no evictions, no demotes. Working set too small per session.
2. **128×6 with conc=32** (the run in §4.2): cache *does* overflow – 192 drops off-pass – but the per-turn hit rate still climbs because the LRU pattern aligns with the workload's access pattern.

A true cliff demonstration needs a workload where requests revisit prefixes AFTER many other prefixes have flushed them. `bench_cache.multiturn` as shipped doesn't do this; a small modification (e.g. round-robin across sessions turn-by-turn, or a second pass that re-asks turn 1 of every session) would. I didn't have time to wire that up cleanly and validate it.

What I *can* show as quantitative evidence the mechanism does the right thing under pressure:

- **GPU eviction count went from 192 -> 0** under HiCache. That's unambiguous; every off-pass drop became an on-pass demote.
- **CPU tier accumulated 193 cached pages** (out of 635 available) at the end of the on-pass run, vs. those same pages being entirely lost in the off-pass.
- **7 promotions happened spontaneously** even on a workload that structurally minimizes re-access – when re-access did occur, HiCache responded correctly.

4.4 Bonus A – `--hicache-overlap` mechanism

Boots cleanly: at startup the engine logs `HiCache: ... pinned=True overlap=True` and `HiCache overlap stream: <torch.cuda.Stream device=cuda:0 cuda_stream=0x...>`. The dedicated stream is created once and reused; pinned host memory is what makes the async DMA non-blocking.

A 384-request smoke (`64×6×192 conc=32 --hicache-overlap`):

	Overlap on
Wall time	108.7 s
Throughput	3.53 req/s
Hit rate	31.3%
Pages demoted	54
`total_demote_time_ms`	552 ms
GPU evictions	0
Pool integrity	`cached+free = 91+7 = 98 = capacity` □

Average issuer-side demote time ≈10 ms per demote; the actual GPU copy time on the copy stream is lower because the issuer doesn't block on completion. The deferred-free queue keeps source pages reserved until their recording event fires; `allocate` will sync on the oldest pending event as a last resort before raising `KVOutOfMemory`. **No errors, no corruption, no deadlocks** observed under the conc=32 load that previously revealed the m3 double-free bug.

What I did *not* do, and would be the cleanest next deliverable for this bonus: an `nsys` timeline screenshot showing copy-stream H2D bars under compute-stream attention/MLP bars during a heavy promote phase, plus a quantified *promote-time-hidden ratio*. The infrastructure (events, pending-free, deferred sync) is in place; capturing the trace and rendering the ratio is mechanical but I ran out of L4 hours before the deadline.

4.5 Bonus B – ≥20% throughput/TTFT win

Not achieved on the workload I ran. Comparing the §4.2 OFF and ON rows directly: - Throughput: 3.74 vs 3.79 req/s -> +1.3% (well within noise). - TTFT p50: 442 vs 434 ms -> +1.8%.

That's not surprising given §4.3: the workload doesn't materially re-access demoted prefixes, so HiCache's avoided-re-prefill savings simply don't trigger. The structural argument is sound – demoting a 1024-token prefix to CPU is ~1–2 ms over PCIe gen4, while re-prefilling those same tokens on 8B is 50–150 ms, so a single avoided re-prefill is a ~50–100× win on that request's TTFT – but you need re-access to claim it.

A workload that would plausibly hit ≥20%: a multi-turn bench where the client deliberately revisits earlier sessions after many newer ones have flushed the cache. I prototyped this mentally but didn't ship a clean implementation.

---

5. What didn't work, and why

- **40 GB CPU tier OOM-killed the server on first boot.** The L4 has 60 GB host RAM and no swap. Pinning 40 GB on top of the Python process, weights staging, and the model put the cgroup over the limit and dmesg showed `out_of_memory: Killed process (python)`. 24 GB safely lands under the ceiling at ~6.5× the GPU pool – short of the spec's ≥10× ideal, but the ratio still demonstrates the tiering.
- **flash-attn ABI drift.** Between milestone 3 and milestone 4 the L4's torch had been updated; the previously-compiled `flash_attn_2_cuda.so` failed with `undefined symbol: _ZN3c105ErrorC2...` at the first chunked prefill. Rebuilding with `pip install --no-build-isolation flash-attn==2.7.4.post1` fixed it. Captured in `project_l4_instance.md`.
- **`bench_cache aiohttp` total timeout is 300 s** and fires per-request if the queue tail waits too long. At concurrency=1 with 256-token outputs and N≥40 requests, the last few queue entries time out and `asyncio.gather` raises before the bench prints its summary – output file ends up empty. Easy fix: use concurrency ≥ 4 for any non-trivial multi-turn run.
- **m3 baseline double-frees.** Already covered in §2. The bug only mattered once load was high enough to trigger eviction; m3 tests passed because no test exercised the post-eviction interaction.
- **Cliff demonstration on vanilla `bench_cache.multiturn`.** Covered in §4.3. Without modifying the bench's access pattern, the LRU/workload alignment hides HiCache's value in the per-turn average even when 192 pages get evicted.

6. Status summary

Deliverable	Status
`--cpu-cache-size-gb` flag, byte-identical to m3 at 0	Done
GPU eviction -> CPU demote (blocking)	Done
Promote-on-hit with temp-lock + rebuild matched_pages	Done
CPU-tier LRU + fall-back drop	Done
`--hicache-overlap` (dedicated CUDA stream + pinned + deferred-free)	Done – boots, runs, pool integrity preserved
`/cache_stats` HiCache counters	Done
Unit test coverage (CPU-only, including bitwise round-trip)	Done 59/59
GPU smoke (L4 multiturn 768 requests, no errors)	Done
Demote/promote mechanism verified end-to-end	Done – 200 demotes, 7 promotes, 0 GPU drops
Cliff/restore per-turn table	Partial – pool overflows and 192 drops are eliminated, but the per-turn average doesn't drop because the workload doesn't re-access
MMLU ±1pp	Not run; correctness argued by token-identity of indexed copies
`nsys` timeline + promote-time-hidden ratio	Not captured
≥20% throughput/TTFT win	Not achieved on this workload

The HiCache surface is built and integrates cleanly; the bonuses needed either a workload with explicit re-access (perf-win) or an `nsys` capture (overlap evidence) that I couldn't quite land before submission.